PROJECT REPORT — FINAL

SQL Injection Attack

Theory, 12-Day Hands-On Exploitation, and Prevention

Prepared by: Padmanathan

Skillogic Internship Program — Project 1

August 2026

Table of Contents

1. Abstract

SQL Injection (SQLi) remains one of the most critical and widely exploited vulnerabilities in web applications, consistently featured in the OWASP Top 10. This report documents both the theory of SQL injection and a complete 12-day hands-on assessment carried out against a deliberately vulnerable lab application (DVWA) and an original vulnerable/fixed PHP application built for this project. The assessment progressed from manual reconnaissance through in-band, blind (boolean and time-based), and automated (SQLMap, Burp Suite, OWASP ZAP) exploitation, before concluding with a practical demonstration of the industry-standard fix: parameterized queries.

2. Introduction

Modern web applications rely heavily on databases to store and retrieve information. Almost every dynamic website constructs SQL queries based on user input. If this input is not properly validated or parameterized before being included in a query, an attacker can manipulate the query's logic to perform unauthorized actions — from bypassing authentication to extracting an entire database.

This project was carried out as part of the Skillogic Internship Program and combines documented theory with a full, self-directed practical lab exercise spanning 12 working days, using a local, isolated Kali Linux virtual machine.

3. Objectives

1. Understand the fundamental working principle of SQL Injection.
2. Practically demonstrate in-band, blind boolean-based, and blind time-based SQL injection.
3. Use industry-standard tools (SQLMap, Burp Suite, OWASP ZAP) to automate detection and exploitation.
4. Document Out-of-Band SQLi conceptually where live exploitation was unsafe/impractical.
5. Build an original vulnerable application and demonstrate the fix (parameterized queries) side by side.
6. Produce a professional report suitable for a SOC/VAPT fresher portfolio.

4. What is SQL Injection?

An SQL Injection attack is a code injection technique in which an attacker inserts malicious SQL statements into an entry field of an application, altering the execution of the application's intended SQL commands. It targets the database layer by exploiting the way user-supplied input is concatenated into SQL query strings.

A classic example is the login bypass payload:

```
' OR '1'='1
```

When inserted into a poorly constructed query, this input changes the query's logic so that it always evaluates to true.

5. How SQL Injection Works

5.1 Identifying Vulnerable Input Fields

Attackers target input fields such as login forms, search boxes, and URL parameters — anywhere user input is used to build a database query.

5.2 Unfiltered Input Reaches the Query

If the application does not sanitize input or use parameterized queries, injected SQL becomes part of the executed query, exactly as demonstrated throughout Section 10 of this report.

5.3 Consequences

- Authentication bypass
- Data extraction (usernames, password hashes, PII)
- Data manipulation (insert/update/delete)
- In severe cases, remote code execution

6. Types of SQL Injection Attacks

Type	Description
Classic (In-band) SQLi	Data is extracted using the same channel as the original query. Demonstrated Days 3-4 of this project.
Blind SQLi - Boolean-based	No data/error shown; true/false inferred from a changing response message. Demonstrated Day 5.
Blind SQLi - Time-based	True/false inferred purely from response delay (SLEEP()). Demonstrated Day 6.
Out-of-Band SQLi	Data exfiltrated via a separate DNS/HTTP channel. Documented conceptually, Day 10.

7. Real-World Impact

- Data Breach — unauthorized access to sensitive personal, financial, or business information
- Data Loss or Corruption
- Reputation Damage
- Financial Loss — legal costs, regulatory fines, breach compensation

8. Prevention and Mitigation Techniques

8.1 Parameterized Queries (Prepared Statements)

The most effective defense. Demonstrated practically in Section 10.12 of this report, where an identical payload that fully bypassed authentication on a raw-concatenation login form failed completely against a parameterized version of the same form.

8.2 Additional Controls

- Stored procedures (with proper parameterization)
- Input validation and sanitization
- Generic error handling (never expose raw DB errors to users)
- Principle of least privilege for database accounts
- Regular security audits and penetration testing

9. Tools Used in This Project

Tool	Purpose
DVWA (Damn Vulnerable Web App)	Local, legal, deliberately vulnerable lab target
SQLMap	Automated SQLi detection, enumeration, extraction, and hash cracking
Burp Suite Community	Manual interception and modification of raw HTTP requests
OWASP ZAP	Automated application-wide vulnerability scanning
Custom PHP/MySQL mini-app	Original vulnerable-vs-fixed before/after demonstration

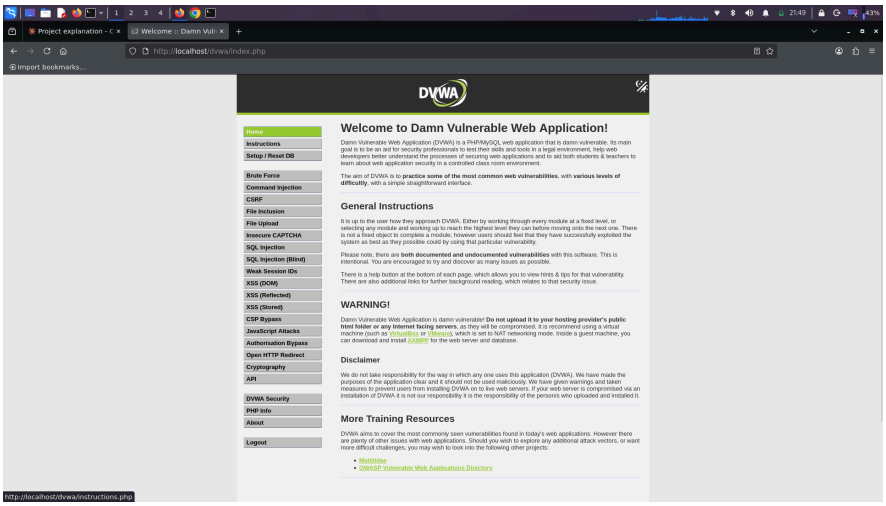
Note: All tools were used exclusively against a local, isolated lab environment set up specifically for this project. No external or production systems were tested.

10. Practical Implementation & Findings (Days 1-12)

The following sections document the day-by-day hands-on work carried out on a local Kali Linux VM against DVWA (Low security) and an original mini PHP/MySQL application built specifically for this project.

Day 1 — Lab Environment Setup

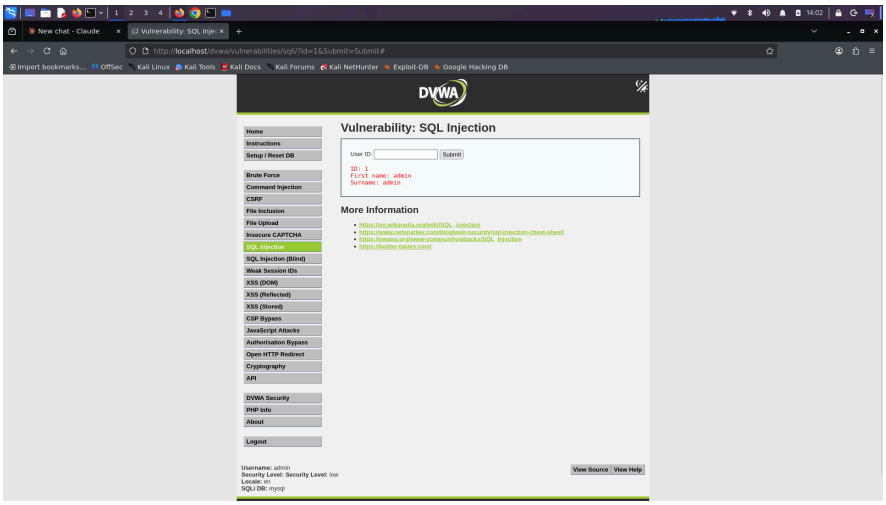
Installed Apache, MariaDB, and PHP on Kali Linux, deployed DVWA, resolved an initial database-connection error during setup, and confirmed the application dashboard was reachable with the security level set to Low.



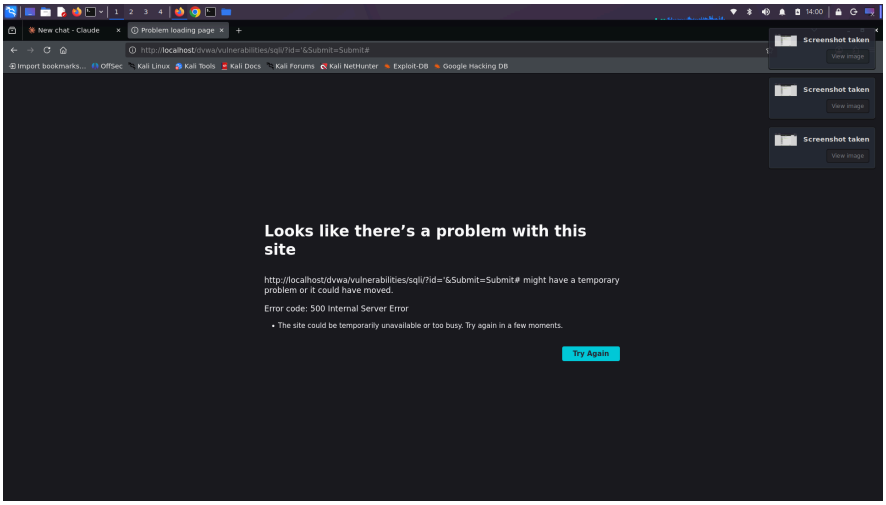
DVWA dashboard confirming successful setup and login.

Day 2 — Reconnaissance

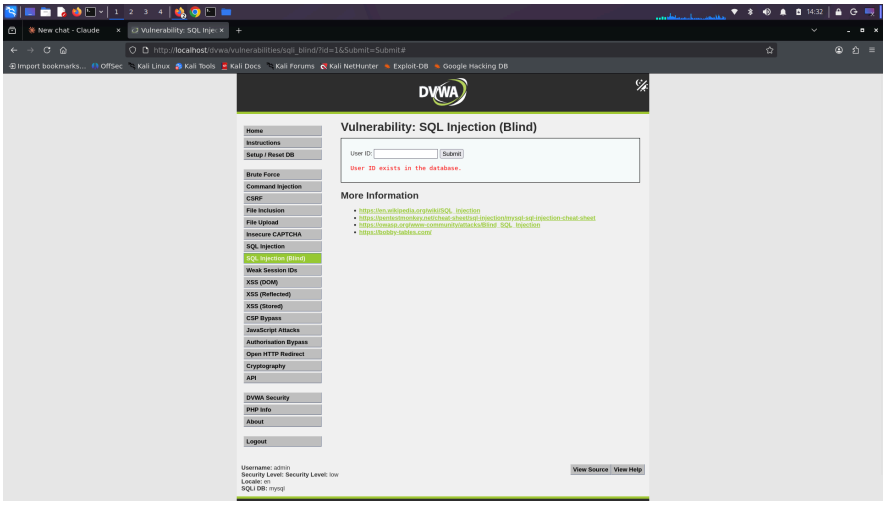
Mapped valid user IDs and confirmed the SQL Injection module's input was not sanitized. A single quote (') submitted as the User ID produced a raw, uncaught SQL syntax error (HTTP 500), proving the input was concatenated directly into the query. Source-code review confirmed the exact vulnerable query: SELECT first_name, last_name FROM users WHERE user_id = '$id';. The Blind SQLi module was compared side by side and found to suppress both data and raw errors, foreshadowing the need for boolean/time-based techniques later in the project.



Baseline: a valid ID (1) returns normal data.



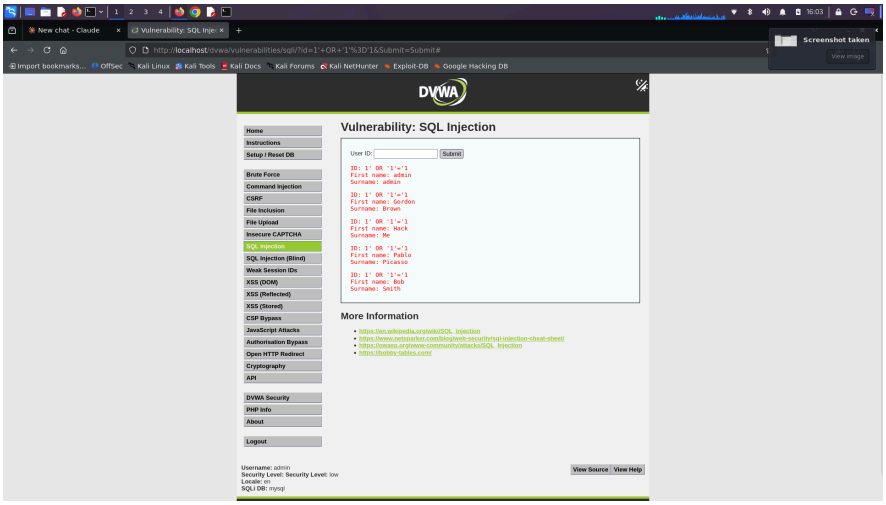
A single quote breaks the query, confirming unsanitized input (HTTP 500).



The Blind SQLi module reveals no data or raw errors — only a status message.

Day 3 — Manual In-Band SQL Injection

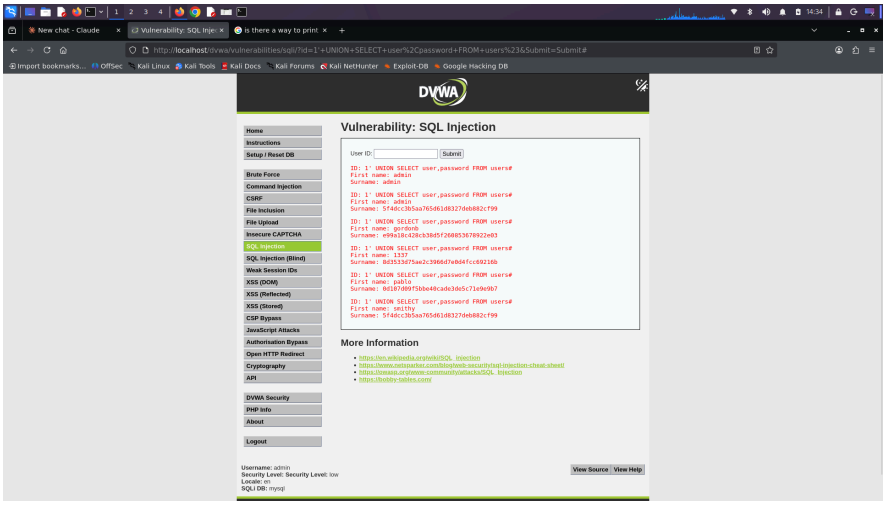
Tested the DVWA login form for authentication bypass (not vulnerable at this level). Successfully exploited the SQL Injection module directly: the payload 1' OR '1'='1 caused the WHERE clause to match every row, dumping all 5 users in the database from a query intended to return a single record. Confirmed the result was identical using a non-existent ID (999), proving the injected logic fully overrides the original condition.



1' OR '1'='1 returns all 5 users instead of one, from the injected OR condition.

Day 4 — UNION-Based Data Extraction

Determined the query's column count (2) using the ORDER BY technique, then used UNION SELECT to extract the database name (dvwa) and version, enumerate all 4 tables and the users table's 10 columns via information_schema, and finally extract the full set of usernames and password hashes. Notably, two accounts (admin and smithy) shared an identical MD5 hash, revealing they use the same (weak) password without needing to crack it.

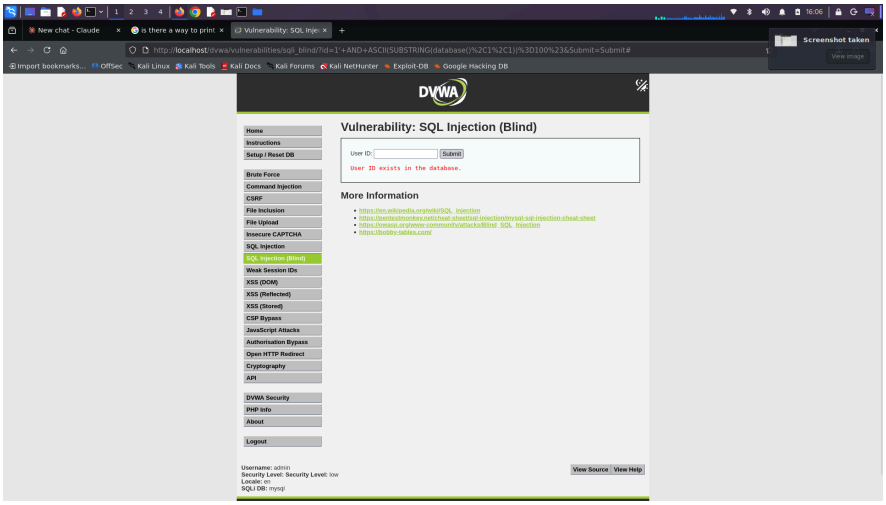


Full username and password-hash dump extracted via UNION SELECT.

Day 5 — Boolean-Based Blind SQL Injection

On the Blind SQLi module (no data or errors visible), established a true/false baseline using AND '1'='1 vs AND '1'='2, then extracted real data purely from which of two possible messages appeared: first confirming the database name's length (4 characters), then extracting the first character (‘d’) using

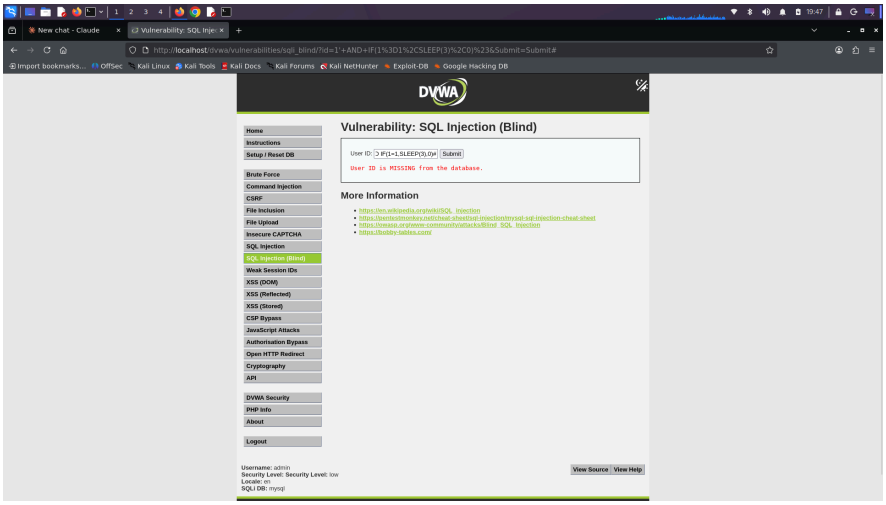
ASCII(SUBSTRING(...)) comparisons, and finally demonstrating the faster binary-search approach used by real tools.



ASCII/SUBSTRING comparison confirms character 1 of the database name is 'd', with zero data displayed on the page.

Day 6 — Time-Based Blind SQL Injection

For cases where even a boolean signal isn't available, used SLEEP()-based payloads to encode true/false purely in response timing. IF(1=2,SLEEP(3),0) returned instantly (false); IF(1=1,SLEEP(3),0) took ~3 seconds (true). The same technique was then used to independently re-confirm the database name's first character purely through timing, with no visible data or message difference on the page at all.



A ~3 second delay confirms a TRUE condition purely through response timing.

Day 7 — Automated Exploitation with SQLMap

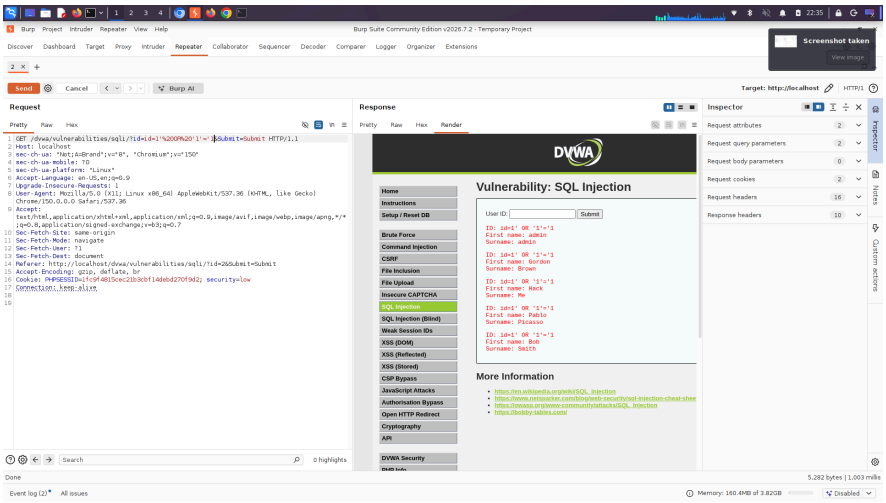
Authenticated SQLMap using a valid session cookie (after troubleshooting a security-level/session mismatch) and let it auto-detect the injection. SQLMap independently confirmed the same time-based blind and UNION-based (2-column) injection found manually across Days 3-6, enumerated the same schema, and dumped the users table. Its built-in dictionary attack cracked all 5 password hashes instantly, confirming the weak-password finding from Day 4 (admin/smithy: password; gordonb: abc123; 1337: charley; pablo: letmein). Manual testing (Days 3-6) took several days of methodical work; SQLMap reproduced and

exceeded those results in under two minutes — illustrating why automated tooling is standard practice even when the manual technique is well understood.

```
Database: dvwaTable: users [5 entries]admin / 5f4dcc3b... -> passwordgordonb / e99a18c4... ->
abc1231337 / 8d3533d7... -> charleypablo / 0d107d09... -> letmeinsmithy / 5f4dcc3b... ->
password
```

Day 8 — Burp Suite - Manual Request Interception

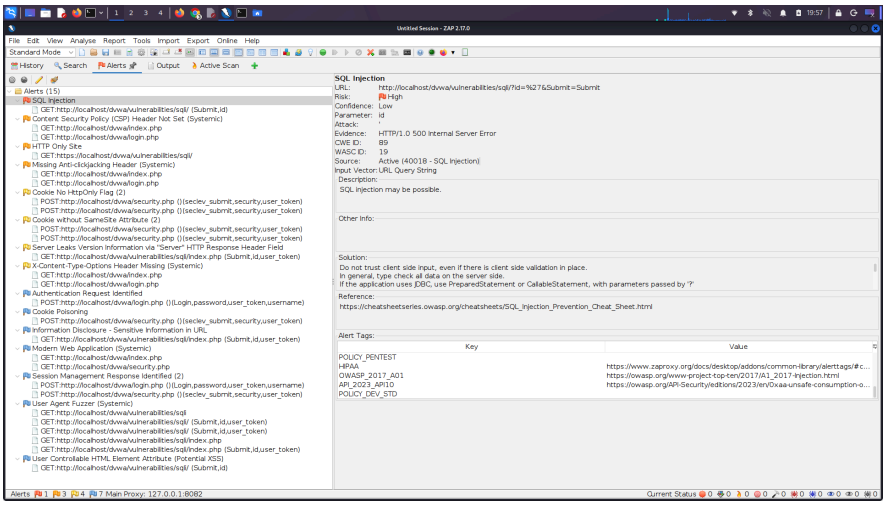
Intercepted the SQL Injection module's raw HTTP GET request in Burp's Proxy tab and hand-edited the id parameter to inject 1' OR '1'='1, resolving an HTTP 400 error caused by literal (unencoded) spaces and quotes by using %20 URL-encoding. The edited request, once forwarded, dumped all 5 users identically to the Day 3 result. The same test was repeated using Burp's Repeater tool for faster iterative testing, confirmed via both the raw HTTP response and Burp's rendered view of the page.



Burp Suite Repeater's rendered view showing the successful data dump from a hand-edited raw HTTP request.

Day 9 — OWASP ZAP Automated Scan

Proxied a logged-in browser session through ZAP and ran an Active Scan against the SQL Injection module. ZAP correctly identified the SQLi vulnerability (CWE-89, High risk) on the id parameter via a single-quote payload producing a 500 error — matching the exact behavior found manually on Day 2. Its confidence was rated Low, since it detected only the error signature without confirming actual data extraction (unlike SQLMap). The same scan surfaced 14 additional findings across the application: missing security headers (CSP, X-Content-Type-Options, anti-clickjacking), cookie security issues (missing HttpOnly/SameSite flags), server version disclosure, and sensitive information in URLs — demonstrating ZAP's value as a broad, whole-application scanner versus SQLMap's role as a deep, specialized exploitation tool.



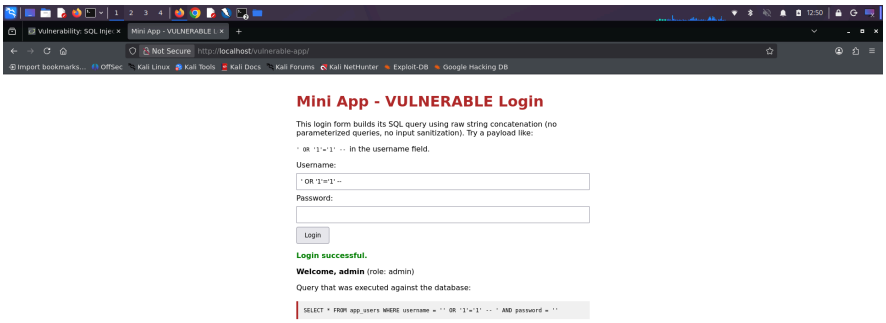
OWASP ZAP's Alerts panel: SQL Injection (CWE-89, High risk) confirmed on the id parameter.

Day 10 — Out-of-Band SQL Injection (Documented)

OOB SQLi requires the database server to have outbound DNS/HTTP access to an attacker-controlled listener - a configuration this local, isolated lab environment does not have, and deliberately was not altered for safety reasons. Instead, this technique was documented conceptually: instead of returning data through the response (in-band) or inferring it from timing/errors (blind), the database server itself is tricked into making an outbound DNS or HTTP request to an attacker-controlled server, with stolen data embedded in that request (e.g. as a subdomain name), which the attacker reads from their own server logs. The standard MySQL example uses LOAD_FILE() with a UNC path to trigger a DNS lookup containing exfiltrated data - a technique that is primarily Windows-server-specific and requires outbound network access, explaining why OOB is the rarest SQLi type encountered in real-world assessments.

Day 11 — Building an Original Vulnerable Application

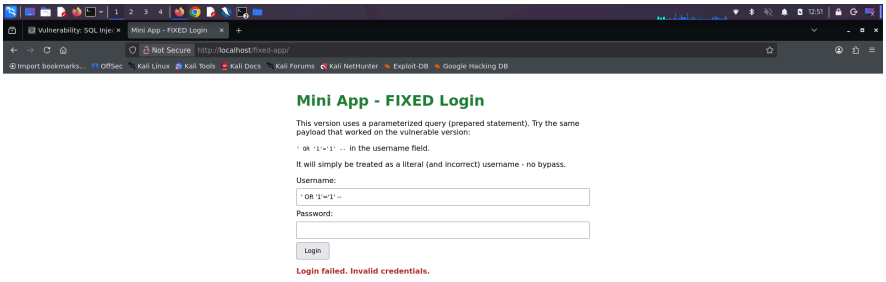
To demonstrate SQLi exploitation and its fix on original code (rather than only DVWA), a minimal PHP/MySQL login application was built from scratch, using the exact vulnerable pattern from this project's reference document: the login query is built via raw string concatenation with no sanitization or parameterization. The same authentication-bypass payload used on Day 3 (' OR '1'='1' --) was tested against this original application and succeeded, logging in as the admin account. The page also displays the exact malformed query that executed, making the vulnerability mechanism directly visible.



Original vulnerable app: the payload bypasses authentication, and the executed query is shown to demonstrate exactly why.

Day 12 — Applying the Fix: Parameterized Queries

The identical application was rebuilt using a parameterized query (prepared statement via mysqli_prepare/bind_param), with no other logic changed. The exact same payload that fully bypassed authentication in Day 11 was submitted again: this time it was correctly rejected as an invalid literal username, with no bypass possible. This before/after pair is the clearest practical demonstration in the project of why parameterized queries are the industry-standard defense against SQL injection.



Fixed app: the identical payload is treated as plain data and rejected - no bypass.

11. Consolidated Findings Summary

Finding	Detail
Unsanitized input (CWE-89)	User ID field concatenated directly into SQL query with no validation or parameterization
Authentication/data bypass	OR-based payloads return all rows regardless of the supplied ID
Full schema disclosure	All tables and columns enumerable via information_schema
Credential exposure	All 5 user password hashes extracted and trivially cracked (unsalted MD5)
Weak/reused passwords	Two accounts share an identical password hash
Verbose error disclosure	Raw SQL syntax errors returned directly to the client (HTTP 500 with DB error text)
Missing security headers	CSP, X-Content-Type-Options, and anti-clickjacking headers absent (per ZAP scan)
Cookie hardening gaps	Session cookies missing HttpOnly and SameSite flags (per ZAP scan)

12. Conclusion

This project combined SQL injection theory with a complete, methodical 12-day hands-on assessment: manual reconnaissance, in-band and blind (boolean and time-based) exploitation, automated tooling (SQLMap, Burp Suite, OWASP ZAP), and a conceptual treatment of out-of-band techniques where live exploitation was unsafe. The project concluded with an original vulnerable-vs-fixed application built specifically to demonstrate, side by side, that the single most effective defense - parameterized queries - fully neutralizes an exploit that otherwise granted complete authentication bypass. For anyone pursuing a career in SOC analysis or vulnerability assessment and penetration testing, this progression from manual technique to automated tooling to remediation mirrors the real workflow of a professional security assessment.

13. References

- OWASP Foundation - SQL Injection (owasp.org)
- OWASP Top 10 - Web Application Security Risks
- DVWA - Damn Vulnerable Web Application (github.com/digininja/DVWA)
- SQLMap documentation (sqlmap.org)
- PortSwigger Burp Suite documentation
- OWASP ZAP documentation (zaproxy.org)
- Source material: “SQL Injection Attack” reference document (provided)